AI Bootcamp — Week 2: Software Development Workflow

Friday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 2, the candidate can:

- Apply the EDD discipline to a domain with right/wrong answers: software development artifacts
- Build out the DLP student-workspace skeleton skills through EC-first iteration, adding Context and Constraints that are demonstrably load-bearing
- Explain the pipeline's YAML file contracts: what each skill reads and writes, the fail-fast behavior on missing inputs, and the review write-back loop
- Direct the AI to write a Python evaluation script against their EC (deterministic checks first, LLM judge second) and run baseline + iteration cycles with it
- Trace quality degradation through a multi-stage pipeline and isolate the source
- Produce an evaluation-measured iteration history showing net improvement across at least one full skill

© 2026 Improving

TIMING

- **Monday–Friday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each day)

© 2026 Improving

FRIDAY (9:00 AM – 5:00 PM): IMPLEMENT STORY, REVIEW IMPLEMENTATION, AND WEEK 2 SYNTHESIS

9:00 – 11:00 AM: IMPLEMENT STORY AND REVIEW IMPLEMENTATION: EDD CYCLES

Apply EDD to both remaining skills. Note that `implement-story` and `review-implementation` are the most context-heavy skills in the pipeline; the quality of their output depends almost entirely on how well the prior skills have defined the story, the AC, and the plan.

Key insight to surface: if implementation quality is low, adding more Constraints to `implement-story` often helps less than improving the quality of `plan-story` output that feeds into it.

The review feedback loop: `review-implementation` does not just produce `review.yaml`. It writes the verdict (`passed` , `failed` , `changes_requested`) and review notes back into `stories.yaml` , so the story carries its own review state. Stories flow forward; review flows back. Run a review and verify the write-back happened.

11:00 AM – 12:00 PM: FULL PIPELINE RUN

Run a complete pipeline from a new, unseen product description (instructor provides a second description). Run all seven skills in sequence. Do not change any skill during this run; this is a test of whether your improvements generalize to a new input.

12:00 – 1:00 PM: LUNCH

1:00 – 3:00 PM: WEEK 2 SYNTHESIS: COMPARISON AND LOAD-BEARING AUDIT

Compare your Week 2 Day 5 pipeline output to your Week 2 Day 1 baseline output on the same product description.

For each skill: how many Constraints did you add that are genuinely load-bearing? How many instructions did the load-bearing audit remove? What did the evaluation scores show across iterations? Where did deterministic checks suffice, and where did you genuinely need the LLM judge?

3:00 – 5:00 PM: REFLECTION AND ARTIFACTS REVIEW

Week 2 Artifacts Review:

- [ ] All 7 skill files committed with completed Context and Constraints sections
- [ ] Hand-written EC for at least `create-prd` and `create-architecture` , written before the skills were touched
- [ ] Python evaluation scripts committed to `evaluations/` , with baseline + iteration results for all 7 skills
- [ ] Load-bearing audit documentation for at least 3 skills
- [ ] Upstream quality dependency notes: at least one documented case where a downstream failure traced back to an upstream skill
- [ ] One complete pipeline run with `stories.yaml` , `plan.yaml` , and `review.yaml` committed, including the review write-back to `stories.yaml`

© 2026 Improving

© 2026 Improving

WEEK 2 DELIVERABLES (COMMITTED TO REPO)

- [ ] All 7 skeleton skill files built out (Context and Constraints complete, all instructions load-bearing)
- [ ] Python evaluation scripts with baseline + iteration results for all 7 skills (deterministic checks plus LLM judge where judgment is required)
- [ ] Load-bearing audit notes for at least 3 skills
- [ ] Side-by-side comparison: Day 1 baseline output vs. Day 5 output for the same product description
- [ ] Upstream dependency trace: at least one documented quality failure traced to its source skill
- [ ] Pipeline artifacts from one full run: `stories.yaml`, `plan.yaml`, `review.yaml`, with the review write-back visible in `stories.yaml`

© 2026 Improving